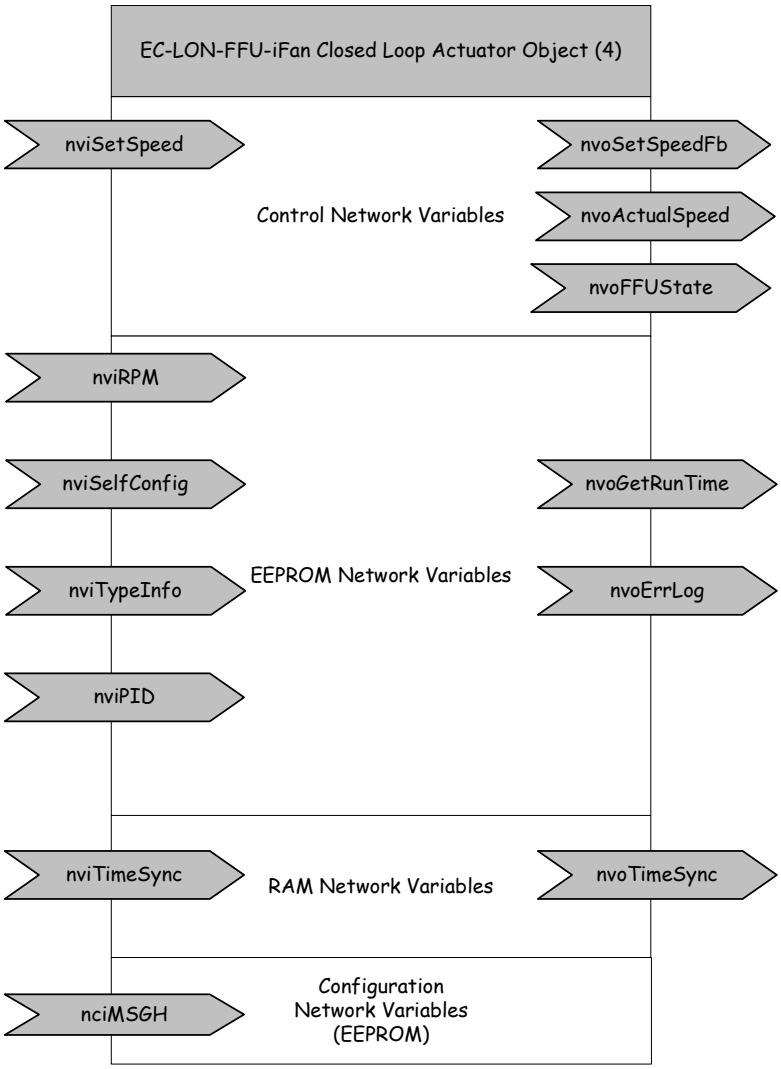
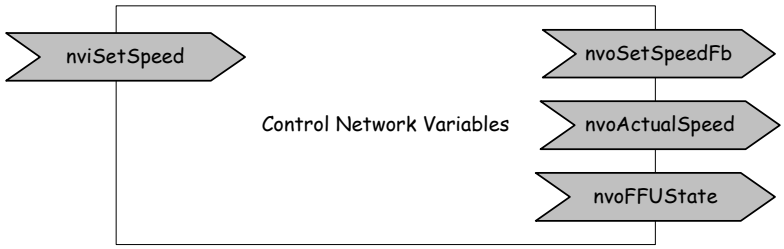
EC-LON-FFU Network Interface Documentation

The network interface of the EC-LON-FFU is designed as a closed loop actuator (LonMark Object #4).

1 Overview EC1xV700



2 Control Network Variables



Declaration:

```
network input  sd_string ("@1|1.") SNVT_rpm nviSetSpeed;
network output sd_string ("@1|2.") SNVT_rpm nvoSetSpeedFb;
network output sd_string ("@1|6.") SNVT_rpm nvoActualSpeed = 0;
network output sd_string
("@1|7.;bit0=Com,bit1=Mot,bit3=Fan,bit4=Temp,bit7=TimeSync,bit8=Wink") SNVT_state nvoFFUState;
```

- **nviSetSpeed:**
  The nviSetSpeed network variable (NV) is used to start and stop the FFU.
  Used values: Start ⇨ nviSetSpeed = 400 to 2050 (rpm)
  Stop ⇨ nviSetSpeed = 0
  This value is stored in the internal Neuron Memory after the following conditions:
  - at Power Shut Down
  - when nviSetSpeed has changed, after about 1h automatically
  - when nviSetSpeed is set to zero
  - when FFU is started (nvoActualSpeed is zero)

Attention: max. 10000 write cycles!

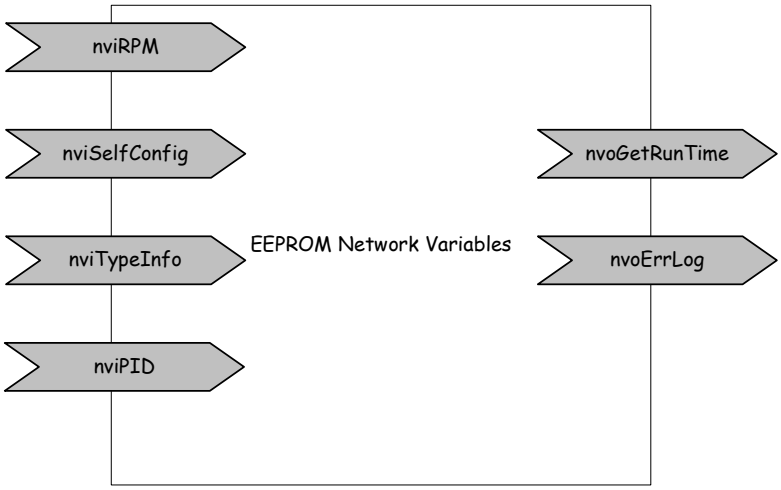
- **nvoSetSpeedFb:**
  This NV is a mirror of nviSetSpeed and is used for the closed loop actuator feedback communication.
  Used values: FFU is turned ON ⇨ nviSetSpeedFb = “rpm” SetSpeed
  FFU is turned OFF ⇨ nviSetSpeedFb = 0

- **nvoActualSpeed:**
  This NV displays the actual rpm.
  Used values: FFU is running ⇨ nvoActualSpeed = “rpm” actual value
  FFU is stopped⇨ nvoActualSpeed = 0

Attention: To avoid heavy traffic on the bus, this variable should only be polled!

- **nvoFFUState:**
  The NV nvoFFUState represents the actual State of the FFU.
  The value is transmitted immediately when its value has changed, except bit7, which is not sent automatically.
  Used values: Communication Error ⇨ bit0 = 1
  Motor blocked ⇨ bit1 = 1
  not used ⇨ bit2, bit5, bit6
  Fan Bad Error ⇨ bit3 = 1
  Temperature Error ⇨ bit4 = 1
  TimeSync⇨ bit7 = 1
  Wink is turned ON ⇨ bit8 = 1
  Remarks: All combinations of the error bits above are possible.

2 EEPROM Network Variables



Declaration:

```
network input  sd_string ("@1|8.") onchip eeprom UNVT_RPM nviRPM ={1304,1304,1304,300};
network input  sd_string ("@1|10.") onchip eeprom UNVT_PID nviPID={10,5,15,48};

network output sd_string ("@1|11.") SNVT_elapsed_tm nvoGetRunTime;
network output sd_string ("@1|15.") UNVT_edate nvoErrLog;
// self initialization at boot time
network input sd_string("@0|3") onchip eeprom SNVT_config_src nviSelfConfig = CFG_LOCAL;
// Type information of the EC-LON-FFU
network input sd_string("@0|4") uninit eeprom SNVT_str_asc nviTypeInfo; // ={K3Gwwyyxx-JJMMDD}
#pragma ignore_notused nviTypeInfo
```

- **nviRPM (manufacture use only)**
  This NV can be used to define the AbsMaxSpeed **ASmax** for the desired wheel diameter **(don´t change this value!)**, the start up speed **Pup** after power shut down, the RelMaxSpeed **RSmax** and the RelMinSpeed **RSmin**.

  ```
  typedef struct {
      unsigned long ASmax;
      unsigned long Pup;
      unsigned long RSmax;
      unsigned long RSmin;
  }UNVT_RPM;
  ```

  Default value: Start up speed ⇨ nviRPM.Pup = 0518h (1304rpm)
  AbsMaxSpeed ⇨ nviRPM.ASmax = 0518h
  RelMaxSpeed ⇨ nviRPM.RSmax = 0518h
  RelMinSpeed ⇨ nviRPM.RSmin = 012Ch (300rpm)

  Used values: 012Ch < nviRPM.Pup < nviRPM.ASmax
  012Ch < nviRPM.ASmax < 0518h
  012Ch < nviRPM.RSmax < nviRPM.ASmax
  nviRPM.RSmin = 012Ch (fix value)

Att: If the ASmax value is changed, the Pup and RSmax value will be set automatically to ASmax value!

If nviSetSpeed<nviPwrUpSpeed → start up speed = nviSetSpeed
If nviSetSpeed>nviPwrUpSpeed → start up speed = nviPwrUpSpeed

All values are in Hex

- **nviPID (manufacture use only):**
  This NV is used to set the P-I-D values of the speed controller (100 → factor 1.0), the Speed Offset for FanBad monitoring (nviPID.So) and to switch the DCI Relay (nviPID.SR).

```
typedef struct {
    unsigned short Kp;
    unsigned short Ki;
    unsigned short Kd;
    unsigned short So;
    unsigned short SR;
}UNVT_PID;
```

Default values:	P value	nviPID.Kp =	0Ah
	I value	nviPID.Ki =	05h
	D value	nviPID.Kd =	0Fh
	SpeedOffset	nviPID.So =	30h
	DCI Relay	nviPID.SR=	00h

Used values:	P:	00h – C8h
	I:	00h – C8h
	D:	00h – C8h
	SpeedOffset:	18h – FAh
	DCI Relay	00h – 01h

All values are in Hex

- **nviSelfConfig (manufacture use only):**
  This network variable can be used to reset the network address to the manufacture address Subnet/Node = 1/1 in the domain = EB, len = 1.
  The default value is CFG_LOCAL. At the very first boot time of the device a special self configuration function is executed and the network address is set to S/N = 1/1. After this execution the NV is set to CFG_EXTERNAL and the self configuration function will NEVER be again executed. However, the self configuration can be repeated using the following sequence:
  Set the value of nciSelfConfig to CFG_LOCAL and perform any reset on the device.
  After this sequence the red LED goes on, indicating the S/N-Address = 1/1.
  Used values: Perform self configuration after reset ⇨ nciSelfConfig = CFG_LOCAL
  Turn off self configuration ⇨ nciSelfConfig = CFG_EXTERNAL

- **nviTypeInfo:**
  This NV can be used to save device specific information. The information includes the used motor, the construction code and the manufacture date of the fan.
  Used values: nviTypeInfo
  Valid range: 30 characters; each 1... 255

- **nvoGetRunTime:**
  This NV represents the actual running time of a FFU. The NV is only updated when the FFU is turned on.
  Remark: The value of the NV will be saved in an internal EEPROM variable every second day and immediately on a power shut down event. If the device is reset, the last saved value will be loaded into nvoGetRunTime.

  Type: SNVT_elapsed_tm

- **nvoErrLog**
  This NV represents the last 4 PSD Errors with date/time, set by nviTimeSync and is saved in internal EEPROM (28 bytes)

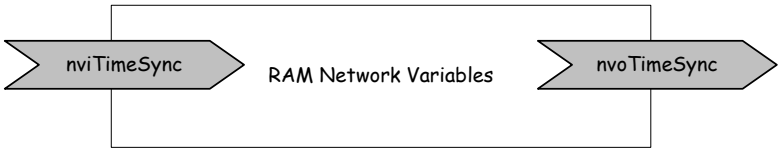
```
typedef struct {
  unsigned short eyear;    (actual year – 2000, eg 2005 – 2000 = 05 (05h)
  unsigned short month;
  unsigned short day;
  unsigned short hour;
  unsigned short minute;
  unsigned short second;
} UNVT_ErrTime;

typedef struct {
    struct{
      unsigned char ecode;
      UNVT_ErrTime err_date;
    }err1;
    struct{
      unsigned char ecode;
      UNVT_ErrTime err_date;
    }err2;
    struct{
      unsigned char ecode;
      UNVT_ErrTime err_date;
    }err3;
    struct{
      unsigned char ecode;
      UNVT_ErrTime err_date;
    }err4;
}UNVT_edate;
```

For an example see the following table for one ErrLog entry :

Err code1	Year	Month	Day	Hour	Min	Sec
0x06	0x04	0x0C	0x04	0x0D	0x26	0x10
PSD	04	12	4	13	38	10

Err code 6: PSD (Input Voltage going below about 160Vac)
Err code 10: PSDON (Input Voltage going above about 170Vac)
Err code 11: PSDMEM (Recalculated PSDON date/time, when a new nviTimeSync is present)
If no year is present or year < 2000, date starts with zero.

4 RAM Network Variables



Declaration:

```
network input  sd_string ("@1|13.") SNVT_time_stamp nviTimeSync;
network output sd_string ("@1|14.") SNVT_time_stamp nvoTimeSync;
```

- **nviTimeSync:**
  This NV is used to set the internal date and clock, to synchronize the time with the PC time.

  If nviTimeSync.year is set to 1, the ErrLog Memory in nvoErrLog is deleted and all fields are set to 0.
  If nviTimeSync.year is set to 2, the status Bit 7 in nvoFFUState is set to 0.

  Type: SNVT_time_stamp

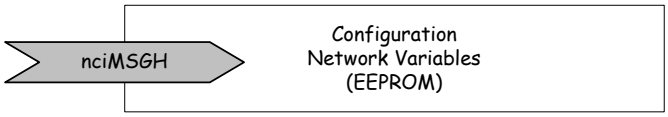
  Default value: 0
  Valid values:

year	0-3000
month	0-12
day	0-31
hour	0-23
minute	0-59
second	0-59

- **nvoTimeSync**
  This NV represents the actual date and time in the FFU

  Type: SNVT_time_stamp

5 Configuration Network Variables (EEPROM)



Declaration:

```
config network input  /*sd_string ("@1|12.")*/ UNVT_MSGH nciMSGH = {255,10,1,0};

typedef struct {
    unsigned short HTSA;
    unsigned short HTNA;
    unsigned short ENHT;
    unsigned short ENPSDHT;
}UNVT_MSGH;
```

- **nciMSGH (manufacturer use only)**
  This NV is used to set the subnet (HTSA) and node (HTNA) address for explicit messaging, enable HT messaging (ENHT) and enable PSD error as Com to HT messaging (ENPSDHT).

  Default value:
  nciMSGH.HTSA = FFh (255)
  nciMSGH.HTNA = 0Ah (10)
  nciMSGH.ENHT = 01h
  nciMSGH.ENPSDHT = 00h

  Valid values:

HTSA:	01h – FFh	
HTNA:	01h – 7Fh	
ENHT:	00h – 01h	1 = Enabled; 0 = Disabled
ENPSDHT:	00h – 01h	1 = Enabled; 0 = Disabled

  All values are in Hex